

IIC 2413 – Bases de Datos Tarea 3 — OLTP vs. OLAP

Información general.

Grupos:	3 personas
Fecha de entrega:	08 de Julio
Tarea OPCIONAL:	La nota reemplaza a la peor tarea
Modalidad:	Informe (PDF) + código reproducible (.zip o repositorio)
Motor OLTP:	PostgreSQL 14+, indexado solo en llaves primarias
Motor OLAP:	DuckDB sobre archivos Parquet
Uso de LLMs:	Permitido y fomentado.

1. Objetivos de aprendizaje

Al completar esta tarea, cada estudiante será capaz de:

1. Derivar, a partir de un mismo diagrama ER, un esquema normalizado (BCNF) para un entorno transaccional y un esquema estrella para análisis.
2. Construir un generador de datos sintéticos parametrizable por tamaño, y un proceso ETL que transforme el esquema OLTP al esquema estrella preservando los datos.
3. Caracterizar una consulta según su *clase* (operación, número de joins, selectividad, agregación) y predecir en qué motor debería ser eficiente.
4. Diseñar y ejecutar un experimento de escalabilidad metodológicamente correcto, distinguiendo ejecución *en frío* y *en caliente*.

2. Contexto

A lo largo del curso vimos dos mundos que conviven en casi toda empresa: el **OLTP** (muchas escrituras y lecturas puntuales, sobre datos vivos) y el **OLAP** (pocas consultas enormes, con agregaciones sobre datos históricos). También vimos que el almacenamiento **por filas** (NSM) y **por columnas** (DSM, Parquet) favorecen cargas de trabajo opuestas.

El objetivo de esta tarea es que ustedes **midan** esa diferencia: construirán *la misma información* en dos motores: PostgreSQL como sistema OLTP y DuckDB sobre Parquet como sistema OLAP y demostrarán experimentalmente que hay clases de consultas en las que uno escala mucho mejor que el otro, y viceversa.

3. Dominio: plataforma de streaming de música

Trabjarán sobre una plataforma de streaming (estilo Spotify). El dominio es fijo, pero el diseño del esquema es de ustedes. La plataforma maneja:

1. **Usuarios.** Cada usuario tiene un identificador, nombre, email, país, fecha de registro y un **plan** de suscripción (**free**, **premium**, **familiar**).
2. **Artistas.** Nombre, país de origen y género musical principal.

3. **Canciones.** Título, artista, duración (en segundos) y género.
4. **Géneros.** Catálogo pequeño de géneros musicales (rock, pop, jazz, ...).
5. **Reproducciones.** El evento central: cada vez que un usuario escucha una canción se registra el usuario, la canción, el `timestamp`, el dispositivo (`movil`, `web`, `smart_tv`) y cuántos segundos efectivamente escuchó. **Esta es la tabla que crecerá a decenas de millones de filas.**

Pueden añadir entidades adicionales (suscripciones/pagos, *playlists*, etc.) si lo necesitan.

4. Entregables

4.1. Parte A — Modelado: ER, BCNF y estrella (15 %)

1. Dibujen un **diagrama ER** del dominio de la sección 3 (entidades, atributos, llaves, cardinalidades).
2. A partir de ese ER, deriven el **esquema OLTP** en **BCNF**, con **al menos 5 tablas**, indicando llaves primarias y foráneas. Justifiquen brevemente por qué está en BCNF.
3. A partir del **mismo** ER, deriven un **esquema estrella**: una tabla de hechos (el evento de reproducción) rodeada de dimensiones desnormalizadas. Indiquen el grano de la tabla de hechos, sus métricas y sus llaves foráneas a dimensiones.
4. Expliquen el **mapeo** entre ambos esquemas: qué tablas OLTP se aplanan en qué dimensión, y de dónde sale cada columna de la tabla de hechos.

4.2. Parte B — Generador de datos y equivalencia BCNF $\leftrightarrow$ estrella (20 %)

1. **Generador sintético.** Escriban un generador parametrizado por un factor de escala s que produzca instancias de tamaño creciente. Fijen una semilla aleatoria para que los datos sean reproducibles. Generen al menos 5 instancias de tamaños crecientes que abarquen al menos 3 órdenes de magnitud en la tabla de reproducciones. La instancia más grande debe cumplir:

- La tabla de **reproducciones** con al menos $\geq 10^7$ filas (orden de decenas de millones).
- **Al menos 3** tablas (p.ej. usuarios, canciones, artistas) con miles de filas.

Los datos deben ser **coherentes**: toda reproducción referencia un usuario y una canción existentes, las duraciones son positivas, `segundos_escuchados` $\leq$ duración de la canción, etc. Pero, pueden ser sintéticos y no hacer sentido (p. ej, fijar los nombres de canciones como 'c1','c2','c3',...).

2. **Carga en ambos motores.** Para cada instancia, pueblen (i) PostgreSQL con el esquema OLTP y (ii) archivos Parquet con el esquema estrella (uno por tabla del esquema estrella), de modo que DuckDB pueda consultarlos.
3. **Transformación OLTP $\rightarrow$ estrella (ETL).** El generador debe crear la instancia OLTP, la instancia estrella debe **derivarse** de la instancia OLTP mediante un proceso ETL documentado (por ejemplo, consultas SQL que desnormalizan las dimensiones y materializan la tabla de hechos, exportadas a Parquet).
4. **Verificación de equivalencia.** Para asegurarse de que esta transformación preserva y no inventa datos, deben usar consultas que se aseguren que, como mínimo, deben coincidir:
 - el número de filas de la tabla de hechos y el de la tabla de reproducciones OLTP;
 - para cada dimensión, el número de entidades distintas (usuarios, canciones, ...);

- la suma de las métricas numéricas principales (p.ej. `SUM(segundos_escuchados)`) calculada en cada esquema.

Este proceso de verificación también debe ser documentado, con un script `verificar_equivalencia` que ejecute estas comprobaciones para cada instancia y reporte PASA/FALLA. **Una instancia que no pasa la verificación no sirve para los experimentos.**

*Sugerencia: generar 10^7 filas con `INSERT` fila a fila es lentísimo. Usen `COPY` (Postgres) o escriban Parquet/CSV en lotes (*pandas*, *polars*, *pyarrow*).*

4.3. Parte C — Clases de consultas y su traducción (20 %)

Definimos una **clase de consulta** como un punto en el espacio de cuatro ejes:

- (1) **Operación:** lectura (`SELECT`) o escritura (`INSERT` / `UPDATE` / `DELETE`).
- (2) **Número de joins:** 0, 1, 2 o 3 o más.
- (3) **Selectividad de los filtros:** *alta* (los filtros y joins dejan pasar muy pocas filas, p.ej. una búsqueda por llave) o *baja* (no hay filtro selectivo; se recorre una fracción grande de una tabla grande).
- (4) **Agregación:** sin agregación (se devuelven o modifican filas crudas) o con agregación y `GROUP BY`.

Una clase queda descrita por una combinación concreta de estos ejes. *Ejemplo: “(SELECT, 3 joins, baja selectividad sobre la tabla grande, con agregación y GROUP BY)”.*

Se les pide:

1. **Elijan 4 clases de consulta distintas.** Deben elegir las de modo que, a priori, esperen que 2 clases favorezcan al motor OLTP y 2 favorezcan al motor OLAP. Para cada clase, describan los cuatro ejes y escriban una hipótesis de qué motor debería ganar y por qué.
2. Para cada clase, escriban 3 consultas concretas que pertenezcan a esa clase, sobre el esquema OLTP. Las 3 consultas deben ser genuinamente distintas (distintas tablas, filtros o agrupaciones).
3. **Traduzcan cada consulta al esquema estrella** (la versión que correrá en DuckDB sobre Parquet).

Las consultas de ambos lados deben tener la misma semántica: sobre datos equivalentes, deben devolver el mismo resultado (salvo, quizá, el orden de las filas).

4.4. Parte D — Experimentos de escalabilidad (30 %)

Deben **demostrar con mediciones** como escalan los motores OLTP y OLAP.

Frío vs. caliente. Toda medición debe reportarse en dos regímenes:

- **En frío (*cold cache*):** la consulta se ejecuta con los datos **fuera de memoria** (ni en el *buffer pool* del motor ni en el *page cache* del sistema operativo). Mide el primer acceso, dominado por I/O de disco. Para lograrlo deben **vaciar las cachés antes de cada medición en frío** (reiniciar el servicio / abrir un proceso nuevo y purgar el *page cache* del SO). Documenten exactamente cómo lo hicieron en su máquina.
- **En caliente (*warm cache*):** la consulta se ejecuta tras haberla corrido al menos una vez, con los datos ya en memoria. Mide el rendimiento sin I/O de disco.

Metodología mínima exigida.

- Midan cada (consulta, motor, tamaño) en frío y en caliente. En caliente, repitan **al menos 5 veces** y reporten la mediana y mín/máx.
- Ejecuten sobre las ≥ 5 instancias de tamaño creciente de la Parte B.
- Pueden fijar un tiempo de timeout y reportar las consultas que lo superan como “no termina”.
- Corran todo en la **misma máquina**, usando la menor cantidad de procesos en paralelo (ideal nada abierto además del sistema operativo). Reporten las specs (CPU, RAM, tipo de disco SSD/HDD) y las versiones de PostgreSQL y DuckDB que usan.

Demostrar escalabilidad. Para cada clase, **grafiquen el tiempo en función del tamaño de la instancia** (si los tiempos crecen mucho, usen una escala log-log), comparando ambos motores en la misma figura. El argumento de escalabilidad no es “en tal tamaño A es más rápido que B”, sino que una de las dos curvas es mucho más sensible al crecimiento del tamaño (su pendiente es mucho mayor). Identifiquen claramente cuáles de sus clases empatan, y en cuales gana OLTP u OLAP.

4.5. Parte E — Informe y reproducibilidad (15 %)

El **informe** (PDF) debe recorrer ordenadamente las Partes A–D: modelado, generador y equivalencia, clases de consulta con hipótesis, experimentos con visualizaciones, y conclusiones. Las conclusiones deben responder explícitamente: ¿qué tipo de consulta conviene en qué motor, y por qué?

- Incluyan las **visualizaciones** (gráficos de tiempo vs. tamaño) en el cuerpo del informe.
- Incluyan **todos los tiempos crudos** (cada repetición) en un **anexo** o archivo de datos adjunto (`.csv`).
- El `.zip`/repositorio debe permitir **regenerar todo** desde cero: generar datos (al menos a escala pequeña), cargar ambos motores, correr los experimentos y reproducir los gráficos, siguiendo instrucciones **completas** en el README.

5. Reglas técnicas y de medición.

1. **PostgreSQL solo con índices de llave primaria.** No creen índices secundarios, ni sobre llaves foráneas, ni *covering indexes*. El único indexado permitido es el que PostgreSQL crea automáticamente para las PRIMARY KEY. (La gracia es comparar un *row store* mínimamente indexado contra un *column store*; un índice secundario en el lugar correcto desvirtuaría el experimento.)
2. **DuckDB consulta Parquet directamente** (vía `read_parquet` o algún comando similar). No construyan índices manuales. Las estadísticas *min/max* por *row group* que trae Parquet **sí** están permitidas, pues son intrínsecas al formato columnar.
3. **Mismos datos en ambos lados.** Los experimentos deben ser sobre instancias que pasan la verificación de equivalencia (Parte B).
4. **Cuidado con el I/O al terminal.** Puede que sea lento escribir los resultados de las consultas al terminal, sobretodo si son muchos. Consideren en vez escribir los resultados a un log.

6. Formato y calendario de entrega

Fecha: 08 de Julio

Entreguen un repositorio (GitHub o .zip) con la siguiente estructura sugerida:

```
tarea3/
├── informe.pdf.....Partes A–E, con visualizaciones
├── modelo/
│   ├── schema_oltp.sql.....Esquema BCNF para PostgreSQL
│   └── schema_estrella.sql.....Esquema estrella / definición de las tablas de hechos y dimensiones
├── generador/.....Generador sintético parametrizado por escala (Parte B)
├── etl/.....Transformación OLTP → estrella + export a Parquet
│   └── verificar_equivalencia.....Script de chequeo de equivalencia
├── consultas/
│   ├── oltp/.....Las 4 clases × 5 consultas, sobre Postgres
│   └── estrella/.....Las traducciones, sobre DuckDB/Parquet
├── experimentos/.....Scripts de medición frío/caliente + generación de gráficos
├── resultados/
│   ├── tiempos_crudos.csv.....Todos los tiempos de todas las repeticiones
│   └── figuras/.....Gráficos tiempo vs. tamaño
└── README.md.....Instrucciones completas de reproducción
```

El README.md debe incluir:

- Nombre y número de alumno de ambos integrantes.
- Comandos completos para: generar datos a una escala chica, cargar ambos motores, correr la verificación de equivalencia, correr los experimentos y reproducir los gráficos.
- Variables de entorno de conexión a PostgreSQL

7. Política de uso de LLMs

El uso de LLMs está **permitido y fomentado**. Completar esta tarea sin usar IA es muy, muy exigente.